Homework 4
Network Layer

Release Date 1404/3/14
Deadline 1403/4/6

Practical Problems

1 Analyzing Protocols Using Wireshark (65 pts)

Introduction to Wireshark

Wireshark is a powerful and widely used network protocol analyzer. It enables users to capture and interactively browse the traffic running on a network, making it an essential tool for understanding and troubleshooting networking issues. Wireshark supports a variety of network protocols, provides detailed packet inspection, and has features such as filtering and live data capture. This makes it an excellent choice for network analysis and debugging tasks.
Make sure to install Wireshark on your system before beginning the tasks and familiarize yourself with its interface using the tutorial provided.

Resources:

- Download Wireshark from Wireshark Official Website.
- For a step-by-step guide on using Wireshark, refer to Wireshark Tutorial by JavaTpoint.

NOTE: Please include enough screenshots from the simulation scenario, and give short and to-the-point answers to the questions (do not write too much!). Save all filtered packets in a separate `.pcapng` file for each section. Submit the `.pcapng` files along with your report.

1.1 Analyze IP Protocol (10)

Goal:
Introduce the IP protocol, examine how IP packets are structured, and understand the meaning of each field in the IP header. This experiment simulates a basic ICMP Echo Request and Reply (ping), which uses the IP protocol to send packets between two hosts (your laptop and your router).

1.1.1 Simulation Scenario: Ping Your Gateway and Analyze IP Packets

1. **Start Wireshark:** Open Wireshark and begin capturing traffic on your active network interface.
2. **Ping the Gateway:** Find your gateway's IP address (e.g., using `ipconfig` or `ifconfig`) and run a ping command (e.g., `ping <gateway_IP>`).
3. **Stop Capturing Traffic:** Stop the capture once the process is complete.
4. **Filter the ICMP Packets:** Use the filter `icmp` in Wireshark to isolate ICMP packets.
5. **Select an ICMP Echo Request Packet:** Expand the Internet Protocol (IP) section in the packet details.

1.1.2 Questions: IP Header Field Analysis

For one captured ICMP Echo Request packet, answer the following:

1. **Header Length:** What is the header length in bytes?
2. **Type of Service (DSCP/ECN):** What value is set? What is the purpose of this field?
3. **Total Length:** What is the size of the entire IP packet?

4. **Identification:** What is the purpose of this field?
5. **Flags and Fragment Offset:** Are any flags set? What do they mean?
6. **Time to Live (TTL):** What is the TTL value? Why is this important?
7. **Protocol:** Which Layer 4 protocol is encapsulated in this IP packet?
8. **Source and Destination IPs:** Do these match your IP and gateway?

1.2 Analyze DHCP Protocol (15)

Goal:
Understand how the Dynamic Host Configuration Protocol (DHCP) works by observing the process of IP address assignment through live packet capture using Wireshark.

1.2.1 Simulation Scenario:

1. Start Wireshark and begin capturing on your active network interface (e.g., Wi-Fi).
2. Release and renew your IP address to initiate DHCP exchange:
   - **Windows:**
     - Open Command Prompt as Administrator.
     - Run: `ipconfig /release`, then `ipconfig /renew`
   - **macOS:**
     - Go to System Preferences → Network.
     - Select your active network interface and click "Advanced".
     - Under the TCP/IP tab, click "Renew DHCP Lease".
3. Let the process run for a few seconds, then stop the Wireshark capture.
4. Apply the following display filter to isolate DHCP packets:

   ```
   dhcp
   ```

5. Identify the DHCP packet sequence: Discover, Offer, Request, Acknowledgment.
6. Select one packet of each type and expand the **Bootstrap Protocol (BOOTP)** section to view the DHCP options.

1.2.2 Questions:

1. What is the MAC address and IP address of your machine as shown in the DHCP packets?
2. Explain the purpose of the packet (e.g., DHCP Discover, Offer, Request, or Acknowledgment).

1.2.3 Simulation Scenario: DHCP Server on Laptop via Mobile Hotspot

In this experiment, your laptop will act as a DHCP server by enabling a mobile hotspot, and your phone will be the DHCP client. You will capture the DHCP exchange on your laptop.

1. Enable Wireshark and begin capturing on the interface used for mobile hotspot (usually "Local Area Connection*" or "vEthernet" on Windows).
2. Enable the mobile hotspot on your laptop:
   - **Windows:** Settings → Network & Internet → Mobile Hotspot.
   - **macOS:** System Settings → Sharing → Internet Sharing.
3. Connect your smartphone to the laptop's hotspot.
4. Wait until your phone connects successfully. During this time, the DHCP Discover-Offer-Request-Ack sequence will occur.

5. Stop capturing after 5–10 seconds.
6. Apply the following Wireshark filter:

   ```
   dhcp
   ```

7. Select each of the DHCP packets and expand the **BOOTP** and **DHCP Options** sections.

1.2.4 Questions:

1. Which device is the DHCP server in this setup? What is its IP and MAC address?
2. Which device is the DHCP client? What IP address is assigned to it?
3. In the `DHCP Discover` packet:
   - What is the source MAC address?
   - What is the destination MAC and IP address?
4. In the `DHCP Offer` packet:
   - What IP address is offered to the phone?
   - What information does the server provide in the DHCP Options (e.g., gateway, DNS)?
5. What is the role of your laptop in this DHCP exchange? Is it acting like a NAT device too?
6. Did you observe any difference between this exchange and the one in the earlier wired/wireless DHCP test? Explain briefly.

1.3 Analyze ARP Protocol (10)

Goal:
Understand the Address Resolution Protocol (ARP), which maps IP addresses to MAC addresses on a local network. Observe ARP request and reply messages using Wireshark.

1.3.1 Simulation Scenario:

1. **Clear ARP Cache:**
   - **For Windows:** Run the command `arp -d *` in the command prompt as administrator.
   - **For Mac:** Run `sudo arp -d -a` in the terminal.
2. **Ping the Gateway:** Find your gateway's IP address (e.g., using `ipconfig` or `ifconfig`) and run a ping command (e.g., `ping <gateway_IP>`).
3. **Filter and Analyze ARP Packets:** Use the filter `arp` in Wireshark to isolate ARP packets.

1.3.2 Questions:

1. Identify the source and destination MAC and IP addresses.
2. Explain the packet's purpose (e.g., ARP request or reply).
3. Open the ARP section in the Packet Details window and describe its fields.
4. Why does the system need to use ARP before sending the first ping packet?
5. Are ARP messages sent as broadcast or unicast? How can you tell from the packet capture?

1.4 Analyze NAT Behavior Using Wireshark (10)

Goal:
Understand how Network Address Translation (NAT) works by observing the IP and port translation that occurs when a mobile device accesses the internet through a laptop hotspot.

1.4.1 Simulation Scenario:

1. Enable the mobile hotspot on your laptop and connect your phone to the laptop's hotspot.
2. Open two instances of Wireshark:
   - One capturing traffic on the **Hotspot Interface** (e.g., Local Area Connection*, or vEthernet).
   - One capturing on the **Wi-Fi or Ethernet Interface** that connects your laptop to the internet.
3. On your phone, go to a speed test website and begin a speed test.
4. Observe packet activity in both Wireshark windows during the test. Then stop both captures after a few seconds.
5. In each capture, apply a display filter such as:

   ```
   ip and tcp
   ```

1.4.2 Questions:

1. What is the IP address of your phone in the hotspot capture? What port did it use to initiate the connection?
2. What is the destination IP and port (i.e., the server performing the speed test)?
3. Compare the IP addresses and ports in both captures. Explain how NAT translated them.
4. Why is NAT needed in this case? Could your phone directly access the internet without the translation?

1.5 Analyze Traceroute Using ICMP (10)

Goal:
Understand how the traceroute utility discovers the path to a destination by examining ICMP Time Exceeded messages.

1.5.1 Simulation Scenario:

1. Start Wireshark and begin capturing traffic on your main internet interface (Wi-Fi or Ethernet).
2. Run the traceroute command to an external IP (e.g., Google DNS):
   - **Windows:** `tracert 8.8.8.8`
   - **macOS/Linux:** `traceroute 8.8.8.8`
3. Let the traceroute complete, then stop the Wireshark capture.
4. Apply the filter:

   ```
   icmp
   ```

5. Identify ICMP Time Exceeded and Echo Reply packets.
6. Examine the source IP of each Time Exceeded packet to identify each router hop.

1.5.2 Questions:

1. How many hops did it take to reach `8.8.8.8`? List the IP addresses of any 3 hops.
2. What type of ICMP messages were generated by intermediate routers?
3. Pick one Time Exceeded packet and describe its source IP, destination IP, and TTL value in the IP header.
4. Why do intermediate routers generate Time Exceeded messages?
5. What happens to the TTL value in each step of the traceroute?
6. Is the final destination's response different from the intermediate hops? What ICMP type/code does it return?

1.6 Analyze IP Fragmentation (10)

Goal:
Explore how IP fragmentation occurs when sending packets larger than the MTU and learn how fragments are identified and reassembled.

1.6.1 Simulation Scenario:

1. Start a new Wireshark capture on your main internet interface.
2. Run a large ping command to generate fragmented packets:
   - **Windows:** `ping -l 2000 <gateway_IP>`
   - **macOS/Linux:** `ping -s <gateway_IP>`
3. Stop capturing after a few seconds.
4. Apply the filter:

   ```
   ip.flags.mf == 1 || ip.frag_offset > 0
   ```

5. repeat

1.6.2 Questions:

1. How many fragments were generated for one large ping request?
2. What is the total length of each fragment? What is the offset of each?
3. What values in the IP header indicate that a packet is fragmented?
4. What is the role of the "More Fragments" (MF) flag and Fragment Offset?
5. If one fragment is lost during transmission, will the full datagram be delivered?
6. What is the Identification field used for in fragmented IP packets?
7. Are fragmented packets reassembled by routers or by the destination device? Explain briefly.

1.6.3 Simulation Scenario:

Repeat the scenario by pinging two different public DNS servers to test fragmentation behavior across different paths:

- `8.8.8.8` – This is a public DNS server operated by Google.
- `1.1.1.1` – This is a public DNS server operated by Cloudflare.

Try both addresses using the same large ping commands. You may observe different behavior (success or timeout) due to variations in how intermediate routers or the destination handle fragmented ICMP packets.

2 Ad-hoc On-Demand Distance Vector Routing (AODV) (35 pts)

Introduction
In this problem, you will learn about the Ad-hoc On-Demand Distance Vector (AODV) protocol and simulate its operation in a network using the INET framework in OMNeT++. Additionally, you will explore INET's visualization tools, configure the PingApp application, and implement mobility in your simulation.

2.1 Preparation

Before starting the tasks, it is essential to review the following examples (just AODV sections) all codes you need to write exists in these examples:

- **AODV Showcase:** MANET Routing Protocols
- **Network Path Activity Visualization:** Visualizing Network Path Activity
- **Routing Table Changes Visualization:** Visualizing Changing Routing Tables

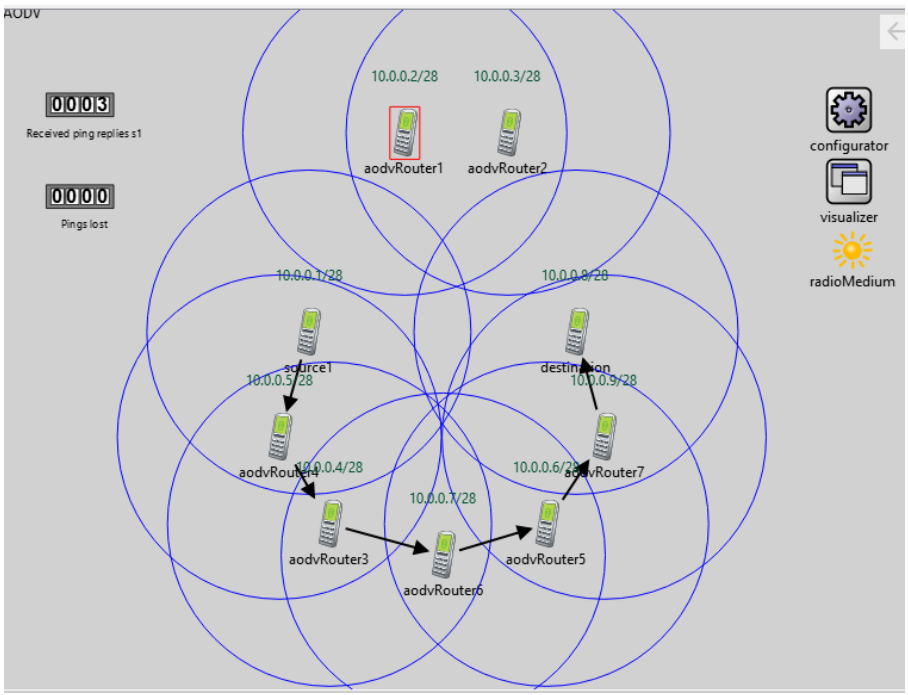
In addition to AODV, you will gain experience with the PingApp and mobility features in INET. For a brief introduction to these concepts, refer to the following documentation:

- **PingApp:** Ping Application Documentation
- **Basic Mobility:** Mobility Examples

2.2 Simulation Setup:

Modify the code of the examples we introduced earlier to create a simulation topology and setup as shown in the image below.

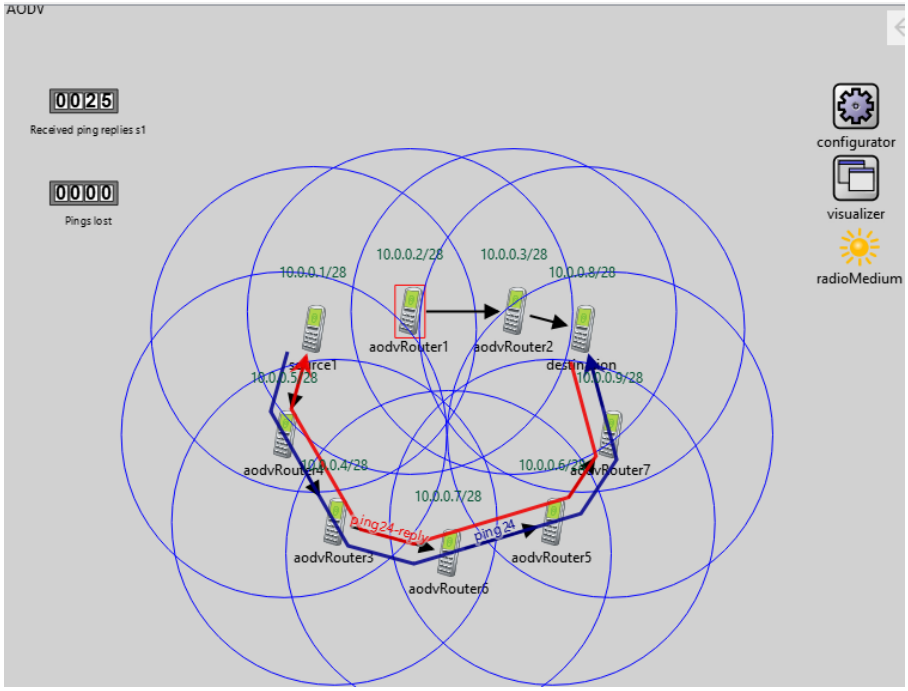
- **Topology Description:** Use the `AODV.ned` file attached to the homework questions to set up AODV routers as shown in the image below.



- **Application Parameters:**
  - Run a `PingApp` on each source node to send packets to the destination.
  - Configure the packet interval to **1 second**.

- Log ping replies and packet loss for analysis.

- **Mobility Configuration:**
  - Use `LinearMobility` for `Router1` and `Router2`.
  - Configure the mobility constraint area to limit node movement vertically with a constant speed (**10 m/s**).
  - Use the following constraints:
    * `*.aodvRouter1.mobility.constraintAreaMinY = 50m`
    * `*.aodvRouter1.mobility.constraintAreaMaxY = 350m`
    * `*.aodvRouter2.mobility.constraintAreaMinY = 50m`
    * `*.aodvRouter2.mobility.constraintAreaMaxY = 350m`



- **Communication Range:**
  - Set the transmitter power of all nodes to **1 mW**.
  - Display communication ranges for all nodes to visualize network connectivity.

- **Visualizer Configuration:**
  - Enable visualization of routing tables, network paths, and data links to better understand protocol dynamics.
  - Include counters to record the number of ping replies received and packets lost for each source.
  - Visualize these metrics during the simulation for real-time monitoring.

2.3 Questions:

1. Explain, with a screenshot, how the path is created and how the first ping packet is sent.
2. explore the AODV router parameters in INET by answering the following:
   (a) Why do we need jitter for broadcasts, and how does it help prevent collisions?
   (b) Why is the initial Time-To-Live (TTL) value set low, and why is it gradually incremented?
   (c) Under what conditions is a route deleted from the routing table, and what parameters influence this process?
3. Analyze the key packet types used in the AODV protocol during the simulation:

(a) **Route Request (RREQ):** Examine the fields in RREQ packets, including: rreqId, hopCount, destSeqNum, ttl.

(b) **Route Reply (RREP):** Review the fields in RREP packets.

(c) What purpose do the Source and Destination Sequence Numbers serve in each routing packet?

4. **`activeRouteTimeout`**

   (a) Does AODV always send packets through the shortest path?

   (b) What is the impact of the `activeRouteTimeout` parameter on the network's ability to select a better path?

   (c) What are the effects of using low and high values for this parameter?

   (d) Simulate the network using different `activeRouteTimeout` values and explain the impact.

5. **Route Error (RERR)**

   - Add mobility to `Router6` in order to break the established route.
   - Observe the scenarios where RERR packets are generated.

6. **Hello Packets:**

   (a) What is the purpose of Hello packets, and why are they disabled by default in some configurations?

   (b) Modify the ping app to observe the effect of hello packets. Configure the ping app to ping 3 times initially. After 3 pings, do not ping again until the destination moves back. Once the destination has moved back, ping again.

   (c) Are Hello packets forwarded like RREQ messages?

NOTE: Include the different stages of AODV with screenshots and in your report.

What Should I Do?

Submit all simulation files (.ini, .ned, .cc, .h) along with your report that includes screenshots of various steps and answers to the related questions. Compress all files and rename the compressed file to DN_HW4_studentID.zip. If you have any questions regarding the problem statement or understanding the concept, feel free to ask.